Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Никитин А.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 17.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Вариант 17) Найти в большом целочисленном массиве минимальный и максимальный элементы

Общий метод и алгоритм решения

Использованные системные вызовы:

- int open(const char **path*, int *flags*, … /* mode_t *mode* */); – открывает файл
- int close(int *fd*); – закрывает файловый дескриптор
- ssize_t read(int *fd*, void *buf*[*count*], size_t *count*); – читает из файлового дескриптора
- ssize_t write(int *fd*, const void *buf*[*count*], size_t *count*); – пишет в файловый дескриптор
- void exit(int *status*); – завершает процесс
- int pthread_create(pthread_t *restrict *thread*, const pthread_attr_t *_Nullable restrict *attr*, typeof(void *(void *_Nullable)) **start_routine*, void *_Nullable restrict *arg*); – создает поток
- int pthread_join(pthread_t *thread*, void ***retval*); – ждет завершения потока
- int gettimeofday(struct timeval **tv*, struct timezone **tz*); – возвращает время

В рамках лабораторной работы была составленна программа, которая находит в целочисленном массиве минимальный и максимальный элементы с использованием заданного пользователем количества потоков. Также была написана вспомогательная программа для генерации массива заданной длины.

Код программы

main.c

```
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>

#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>

#include <unistd.h>
```

```
#include <pthread.h>

#include <sys/time.h>

typedef struct {
       size_t number;
       size_t size;
       size_t n_threads;
       char* filename;
    int* arr;
    bool verbose;
    int min;
    int max;
} ThreadArgs;

static void* work(void* _args)
{
       ThreadArgs* args = (ThreadArgs*)_args;

    int min = INT_MAX;
    int max = INT_MIN;
    size_t len = args->size * sizeof(int);
    size_t x = args->number * (args->size / args->n_threads);
    size_t y = (args->number == args->n_threads - 1) ? args->size : (args->number + 1)
* (args->size / args->n_threads);
    for (size_t i = x; i < y; i++) {
        if (args->arr[i] < min) {
            min = args->arr[i];
        }
        if (args->arr[i] > max) {
            max = args->arr[i];
        }
    }

      if (args->verbose) {
        printf("Thread #%ld says min-max is: %d-%d\n", args->number, min, max);
    }

    args->min = min;
    args->max = max;

      return NULL;
}

int main(int argc, char** argv)
{
       if (argc <= 3) {
              char msg[1024];
              uint32_t len = snprintf(msg, sizeof(msg), "usage: %s filename array-size
number-of-threads [-v]\n", argv[0]);
```

```
        write(STDERR_FILENO, msg, len);
        exit(EXIT_SUCCESS);
  }
bool verbose = argc > 4 && argv[4][0] == '-' && argv[4][1] == 'v';

  size_t n_threads = atoi(argv[3]);

  pthread_t *threads = malloc(n_threads * sizeof(pthread_t));
  ThreadArgs *thread_args = malloc(n_threads * sizeof(ThreadArgs));

ssize_t bytes;
size_t len = atoi(argv[2]) * sizeof(int);
int* arr = (int*)malloc(len);
int32_t file = open(argv[1], O_RDONLY);
  bytes = read(file, arr, len);
close(file);


struct timeval start, end;
gettimeofday(&start, NULL);

  for (size_t i = 0; i < n_threads; ++i) {
        thread_args[i] = (ThreadArgs){
              .number = i,
              .size = atoi(argv[2]),
              .n_threads = n_threads,
              .filename = argv[1],
       .arr = arr,
       .verbose = verbose,
        };

        pthread_create(&threads[i], NULL, work, &thread_args[i]);
  }

  for (size_t i = 0; i < n_threads; ++i) {
        pthread_join(threads[i], NULL);
  }

int min = INT_MAX;
int max = INT_MIN;
for (size_t i = 0; i < n_threads; i++) {
    if (thread_args[i].min < min) {
        min = thread_args[i].min;
    }
    if (thread_args[i].max > max) {
        max = thread_args[i].max;
    }
}
gettimeofday(&end, NULL);
```

```
    double time_taken = end.tv_sec + end.tv_usec / 1e6 - start.tv_sec - start.tv_usec / 1e6;

    printf("Min-max: %d-%d\n", min, max);
    printf("Time: %f\n", time_taken);

    free(arr);
      free(thread_args);
      free(threads);
}
```

genarray.c

```
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>

#include <stdlib.h>
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>
#include <pthread.h>

int main(int argc, char **argv)
{
      if (argc <= 2) {
            char msg[1024];
            uint32_t len = snprintf(msg, sizeof(msg), "usage: %s filename array-size\n", argv[0]);
            write(STDERR_FILENO, msg, len);
            exit(EXIT_SUCCESS);
      }

    srand(time(NULL));
    int32_t file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0600);
    size_t len = atoi(argv[2]) * sizeof(int);
    int* buf = (int*)malloc(len);

    int i = 0;
    for (; i < atoi(argv[2]); i++) {
        int value = rand();
        buf[i] = value;
    }

    write(file, buf, len);
    close(file);

    printf("An array of %d items has been generated\n", atoi(argv[2]));
}
```

run.sh

```
file="array"
len=$1

cc main.c -pthread -o main 2> /dev/null
cc genarray.c -o genarray 2> /dev/null

./genarray $file $len

echo
echo "1 thread:"
./main $file $len 1

echo
echo "2 threads:"
./main $file $len 2

echo
echo "4 threads:"
./main $file $len 4

echo
echo "6 threads:"
./main $file $len 6

echo
echo "12 threads:"
./main $file $len 12

echo
echo "32 threads:"
./main $file $len 32

echo
echo "1024 threads:"
./main $file $len 1024

echo
echo "8192 threads:"
./main $file $len 8192
```

Протокол работы программы

Тестирование

```
$ ./run.sh 100000000
An array of 100000000 items has been generated

1 thread:
Min-max: 36-2147483638
Time: 0.106655

2 threads:
Min-max: 36-2147483638
Time: 0.059015

4 threads:
Min-max: 36-2147483638
Time: 0.035934

6 threads:
Min-max: 36-2147483638
Time: 0.032822

12 threads:
Min-max: 36-2147483638
Time: 0.022689

32 threads:
Min-max: 36-2147483638
Time: 0.017238

1024 threads:
Min-max: 36-2147483638
Time: 0.041900

8192 threads:
Min-max: 36-2147483638
Time: 0.294315
```

Результаты

Используемые формулы:

Ускорение: S = Ts / Tp, где

- Ts – время последовательной реализации,
- Tp – время параллельной реализации,
- p – количество потоков

Ускорение показывает во сколько раз параллельная программа быстрее последовательной

Эффективность: X = S / p, где X < 1

Эффективность показывает насколько эффективна параллелизация

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	106.65	1.00	1.00
2	59.01	1.80	0.90
4	35.93	2.97	0.74
6	32.82	3.25	0.54
12*	22.68	4.70	0.39
32	17.23	6.19	0.19
1024	41.90	2.54	0.0025
8192	294.43	0.36	0.00004

*Количество логических ядер в тестирующей системе

Вывод

Из таблицы результатов видно, что максимальное ускорение было получено в тесте с бóльшим количеством потоков, чем логических ядер. Это можно объяснить тем, что при работе с большими данными немалая часть времени уходит на ожидание получения данных из памяти. В том случае, когда потоков больше логических ядер, выгода от параллельных запросов к памяти может перевешивать накладные расходы на переключение контекста. При сильном увеличении количества потоков накладные расходы могут вырасти настолько, что программа будет работать в разы медленнее, чем в однопоточном режиме, что можно увидеть в таблице.